

UNIVERSITY OF TARTU
Institute of Computer Science
Informaatika õppekava

Uku Renek
**Suurkeelemudelite kasutamine automaatses
koodiülevaatuses**
Bakalaureusetöö (9 EAP)

Supervisor:
Prof. Vanem Juhendaja, PhD

Tartu 2025

Suurkeelemudelite kasutamine automaatses koodiülevaatuses

Abstract:

Koodiülevaatus on tarkvaraarenduse oluline kvaliteeditagamise etapp, mille käigus kontrollivad arendajad üksteise lähtekoodi vigade, turvaprobbleemide ja stiilireeglite rikkumiste leidmiseks. Kuigi manuaalne koodiülevaatus on tõhus, on see ressursimahukas ja ajakulukas protsess, mis sõltub ülevaatajate kogemusest ja tähelepanelikkusest. Suurkeelemudelid (ingl *large language models*, LLM) on viimastel aastatel näidanud lubavaid tulemusi loomuliku keele töötlemisel ja koodi mõistmisel, kuid nende rakendatavust automaatses koodiülevaatuses ei ole süstemaatiliselt uuritud.

Käesolev bakalaureusetöö uurib suurkeelemudelite — eelkõige GPT-4, Claude 3 Opus ja Code Llama 70B — sobivust automaatse koodiülevaatuses tööriistadena. Töö eesmärk on hinnata nimetatud mudelite täpsust erinevat tüüpi koodivigade (loogiline viga, turvaprobbleem, jõudlusprobbleem ja stiilirikkumine) tuvastamisel ning võrrelda nende jõudlust nii omavahel kui ka traditsiooniliste staatilise analüüsi tööriistadega. Uurimistöö käigus koostati 120 koodifragmendist koosnev testkogum, mis hõlmas nelja vigade tüüpi Python ja Java keeles. Iga koodifragmenti analüüsisid kõik kolm LLM-i standardiseeritud viipade abil.

Tulemused näitavad, et GPT-4 saavutas kõrgeima vigade tuvastamise täpsuse (83,3%), ületades Claude 3 Opuse (79,2%) ja Code Llama 70B (71,7%). Turvaprobbleemide tuvastamisel erines mudelite jõudlus märkimisväärselt: GPT-4 suutis tuvastada 91,7% turvavigadest, samas kui Code Llama 70B tuvastas vaid 58,3%. Kõik mudelid edestavad traditsioonilisi staatilise analüüsi tööriistu loomuliku keele seletuste ja kontekstuaalse tagasiside poolest. Töö järeldab, et suurkeelemudelid sobivad olemasolevate koodiülevaatuses protsesside täiendamiseks, kuid mitte täielikuks asendamiseks, arvestades nende hallutsinatsiooni kalduvust ja puuduvat koodi täitmise konteksti.

Keywords: suurkeelemudelid, koodiülevaatus, tarkvaraarendus, staatiline analüüs, GPT-4, Claude, Code Llama

CERCS: P170 Arvutiteadus, arvutusmeetodid, süsteemid, juhtimine (automaatjuhtimisteooria)

Large Language Models for Automated Code Review

Abstract:

Code review is an essential quality assurance step in software development, during which developers examine each other's source code to identify bugs, security vulnerabilities, and style violations. While manual code review is effective, it is a resource-intensive and time-consuming process that depends on reviewers' experience and attentiveness. Large language models (LLMs) have demonstrated promising results in natural language processing and code comprehension in recent years, yet their applicability to automated code review has not been systematically studied.

This bachelor's thesis investigates the suitability of large language models—specifically GPT-4, Claude 3 Opus, and Code Llama 70B—as automated code review tools. The aim is to evaluate the accuracy of these models in detecting different types of code defects (logic errors, security vulnerabilities, performance issues, and style violations), and to compare their performance both with each other and with traditional static analysis tools. A test dataset of 120 code snippets covering four defect types in Python and Java was compiled. Each snippet was analyzed by all three LLMs using standardized prompts.

Results show that GPT-4 achieved the highest defect detection accuracy (83.3%), surpassing Claude 3 Opus (79.2%) and Code Llama 70B (71.7%). Performance varied significantly for security vulnerability detection: GPT-4 identified 91.7% of security bugs, while Code Llama 70B detected only 58.3%. All models outperform traditional static analysis tools in terms of natural language explanations and contextual feedback. The thesis concludes that large language models are suited to supplement existing code review processes, but not to fully replace them, given their tendency to hallucinate and their lack of runtime execution context.

Keywords: large language models, code review, software development, static analysis, GPT-4, Claude, Code Llama

CERCS: P170 Computer science, numerical analysis, systems, control

Contents

1. Sissejuhatus	7
1.1 Probleemi kirjeldus	7
1.2 Uurimiseesmärgid ja uurimisküsimused	8
1.3 Töö ülesehitus	8
2. Teoreetiline taust.....	9
2.1 Koodiülevaatus tarkvaraarenduses.....	9
2.1.1 Koodiülevaatuse olemus ja eesmärgid.....	9
2.1.2 Koodiülevaatuse protsess	9
2.1.3 Koodiülevaatuse katsumused	10
2.2 Automatiseeritud koodianalüüs.....	10
2.2.1 Staatilise analüüsi tööriistad.....	10
2.2.2 Masinõppepõhised lähenemised.....	10
2.3 Suurkeelemudelid	11
2.3.1 Transformeri arhitektuur.....	11
2.3.2 Eeltreenimine ja finantreenimine.....	11
2.3.3 Peamised suurkeelemudelid	12
2.4 LLM-id tarkvaraarenduses.....	13
2.4.1 Koodi genereerimine	13
2.4.2 Vigade parandamine	13
2.4.3 Koodi mõistmine ja dokumenteerimine	13
2.4.4 Koodiülevaatus LLM-ide abil.....	13
3. Seotud tööd	15
3.1 LLM-id koodiülevaatuse kommentaaride genereerimisel	15
3.2 Koodiülevaatuse automatiseerimine LLM-ide abil.....	15
3.3 GPT mudelite kasutamine koodi analüüsimisel	16
3.4 Code Llama ja avatud LLM-ide jõudlus	16
3.5 Staatilise analüüsi tööriistad ja automaatne koodiülevaatus	16
3.6 Kokkuvõte ja lüngad	17
4. Metoodika	18
4.1 Uurimisplaan	18
4.2 Testkogumi koostamine	18
4.2.1 Vigade tüüpide valik.....	18

4.2.2	Koodifragmentide koostamine.....	19
4.2.3	Annotatsiooniprotsess.....	19
4.3	Kasutatavad mudelid ja nende seadistus	19
4.3.1	GPT-4.....	19
4.3.2	Claude 3 Opus.....	20
4.3.3	Code Llama 70B.....	20
4.4	Viipade disain	20
4.4.1	Standardne viip	20
4.4.2	Väljundi töötlemine.....	20
4.5	Hindamiskriteeriumid	21
4.6	Võrdlus staatilise analüüsi tööriistaga	21
4.7	Valiidsus ja usaldusväarsus	22
4.7.1	Sisemine valiidsus.....	22
4.7.2	Väline valiidsus.....	22
5.	Tulemused.....	23
5.1	Üldised tulemused	23
5.2	Vigade tüüpipõhine analüüs	23
5.2.1	Loogilised vead	23
5.2.2	Turvaaugud	24
5.2.3	Jõudlusprobleemid	24
5.2.4	Stiilirikkumised	25
5.3	Programmeerimiskeeleline analüüs	25
5.4	Vastuste kvalitatiivne analüüs.....	26
5.4.1	Seletuste kvaliteet	26
5.4.2	Valepositiivsete analüüs	26
5.5	Eksperimendi kulud ja kiirus.....	26
6.	Arutelu	28
6.1	Peamiste leidude interpretatsioon.....	28
6.1.1	GPT-4 üldine ülekaalukus.....	28
6.1.2	Code Llama kui alternatiiv.....	28
6.1.3	LLM-id vs staatilise analüüsi tööriistad	28
6.2	Praktilised implikatsioonid	29
6.2.1	Koodiülevaatuse töövoo integreerimine.....	29

6.2.2	Mudelivalik kontekstist sõltuvalt.....	29
6.2.3	Viibade optimeerimine.....	29
6.3	Uurimistöö piirangud.....	30
6.3.1	Testkogumi piirangud	30
6.3.2	Mudelite piirangud.....	30
6.3.3	Hallutsinatsioon.....	30
6.4	Edasised uurimissuunad	30
7.	Kokkuvõte.....	32
7.1	Uurimisküsimuste kokkuvõte	32
7.2	Peamised järeldused	33
7.3	Töö panus	33
	References.....	34
	Lisad	37
	Litsents	40

1. Sissejuhatus

Kaasaegne tarkvaraarendus on olemuselt meeskonnatöö. Isegi suurima kogemusega programmeerijad teevad vigu, ning keerukad süsteemid sisaldavad sageli turvaauke, jõudlusprobleeme või loogilisi vigu, mida üks inimene ei suuda leida. Koodiülevaatus (ingl *code review*) on sellele probleemile vastuseks kujunenud tarkvaratehnika praktika, kus üks või mitu arendajat kontrollivad teise arendaja kirjutatud lähtekoodi enne selle integreerimist põhiharusse [1]. Uuringud on näidanud, et süstemaatiliselt rakendatud koodiülevaatus vähendab vigade arvu lõpptootes, parandab koodi arhitektuuri ning soodustab teadmiste jagamist meeskonna liikmete vahel [2].

Koodiülevaatus on eriti ilmne suurtes tarkvaraarendusprojektides. Google'i andmetel vaatab nende inseneritiim läbi igapäevaselt tuhandeid koodimuudatusi, kusjuures iga muudatus peab saama vähemalt ühe heakskiidu enne integreerimist [3]. Microsoft ja Meta järgivad sarnaseid protseduure. Selline mastaap seab ülevaatajatele tohutuid nõudmisi: iga ülevaatus jaoks tuleb eraldada aega, mis on paratamatult ära võetud uute funktsioonide arendamiselt. Bacchelli ja Birdi uuringus leidsid arendajad, et koodiülevaatus võtab keskmiselt 6–10% tööajast [1].

1.1 Probleemi kirjeldus

Manuaalsel koodiülevaatusel on mitmeid fundamentaalseid piiranguid. Esiteks on see *inimfaktori-sõltuv*: ülevaataja väsimus, tähelepanu hajumine või puudulikud teadmised teatud valdkonnas võivad põhjustada oluliste vigade märkamata jätmist. Teiseks on see *skaleerumisprobleemiga* protsess: koodibaasi kasvades kasvab ka ülevaatus koormus, kuid ülevaatajate arv ei kasva samas tempos. Kolmandaks on manuaalne koodiülevaatus *ebaühtlane kvaliteedi poolest* — sama koodifragment võib saada erinevaid hinnanguid eri ülevaatajatelt.

Nende probleemide leevendamiseks on tarkvaratööstus pöördunud automatiseerimise poole. Staatilise analüüsi tööriistad nagu PMD, SonarQube ja Checkstyle suudavad automaatselt tuvastada teatud tüüpi koodi kvaliteediprobleeme, kuid nende piirangud on hästi teada: need tööriistad tuginevad eeldefineeritud reeglitele, ei suuda mõista konteksti ning tekitavad sageli suure hulga valepositiivseid tulemusi [3].

Viimastel aastatel on tehisintellekti, eelkõige suurkeelemudelite (ingl *large language models*, LLM) areng avanud uusi võimalusi tarkvaraarenduse automatiseerimisel. GPT-4 [4], Claude 3 [5] ja avatud lähtekoodiga Code Llama [6] suudavad mõista ja genereerida lähtekoodi mitmes

programmeerimiskeeles. GitHub Copilot on näidanud, et LLM-id suudavad aidata arendajaid koodi kirjutamisel reaalses arenduskeskkonnas [7]. Küsimus, kas samad mudelid suudavad olla tõhusad ka koodiülevaatuses ehk olemasoleva koodi vigade leidmisel, on suureneva tähtsusega ning vajab empiirilist uurimist.

1.2 Uurimiseesmärgid ja uurimisküsimused

Käesoleva bakalaureusetöö peamine eesmärk on hinnata suurkeelemudelite sobivust automaatse koodiülevaatuses tööriistadena ning võrrelda eri mudeleid nende tulemuslikkuse alusel. Töö käigus viiakse läbi empiiriline katse, milles kolme LLM-i — GPT-4, Claude 3 Opus ja Code Llama 70B — rakendatakse 120 koodifragmendi analüüsimiseks.

Töö uurib järgmisi uurimisküsimusi:

UK1. Milline on LLM-ide täpsus erinevat tüüpi koodivigade tuvastamisel?

UK2. Kuidas erineb eri LLM-ide jõudlus koodiülevaatuses ülesannetes?

UK3. Milliseid vigade tüüpe suudavad LLM-id kõige paremini ja kõige halvemini tuvastada?

UK4. Kuidas võrreldakse LLM-ide automaatne koodiülevaatus traditsiooniliste staatilise analüüsi tööriistadega?

1.3 Töö ülesehitus

Käesolev töö on üles ehitatud järgmiselt. Peatükk 2 tutvustab koodiülevaatuses protsessi, suurkeelemudelite teoreetilist tausta ning nende rakendusi tarkvaraarenduses. Peatükk 3 annab ülevaate seotud töödest ja varasematest uuringutest LLM-ide kasutamisest tarkvara analüüsis. Peatükk 4 kirjeldab uurimismetoodikat, sealhulgas testkogumi koostamist, viipade disaini ja hindamiskriteeriumeid. Peatükk 5 esitab eksperimendi tulemused struktureeritud kujul. Peatükk 6 arutleb tulemuste tähenduse ja piirangute üle. Peatükk 7 võtab töö kokku ja toob välja edasised uurimissuunad.

2. Teoreetiline taust

Käesolev peatükk annab ülevaate kahest põhivaldkonnast, mille ristumiskohas käesolev uurimistöö asetseb: koodiülevaatuses tarkvaraarenduses ning suurkeelemudelitest tehisintellekti valdkonnas. Lisaks käsitletakse LLM-ide varasemaid rakendusi tarkvaraarenduse kontekstis.

2.1 Koodiülevaatus tarkvaraarenduses

2.1.1 Koodiülevaatus olemus ja eesmärgid

Koodiülevaatus on tarkvaraarenduse protsess, mille käigus üks või mitu arendajat (ülevaatajad) loevad ja analüüsivad teise arendaja (autori) kirjutatud lähtekoodi [1]. Kuigi koodiülevaatus põhiprintsiip on lihtne, on selle eesmärgid mitmekihilised.

Esmane eesmärk on *vigade leidmine*: ülevaatajad püüavad tuvastada loogilisi vigu, äärejuhtumeid (ingl *edge cases*), mida kood ei käsitle, ning muid funktsionaalseid probleeme. Teiseks oluliseks eesmärgiks on *turvalisuse tagamine*: koodi analüüsimisel otsitakse tüüpilisi turvaauguparameetreid nagu SQL-i süstimine, puhvri ületäitumine või ebapiisav sisendi valideerimine. Kolmandaks eesmärgiks on *koodi kvaliteedi parandamine*: ülevaatajad kommenteerivad liiga keerukaid konstruktsioone, ebasobivaid nimistandardeid või halvasti struktureeritud koodi. Neljandaks eesmärgiks on *teadmiste jagamine*: koodiülevaatus on vahend, mille kaudu kogenumad arendajad jagavad teadmisi noorematega ning meeskond säilitab ühise arusaama koodi struktuurist [1, 3].

2.1.2 Koodiülevaatus protsess

Kaasaegne koodiülevaatus protsess on tavaliselt integreeritud versioonikontrolli töövoogu. Kõige levinumal mudelil on järgmised sammud. Esiteks loob arendaja koodimuudatuse ja esitab selle läbivaatamiseks (ingl *pull request* või *merge request*). Teiseks määratakse automaatselt või käsitsi ülevaatajad, kellel palutakse muudatust hinnata. Kolmandaks loevad ülevaatajad koodi, lisavad kommentaare ja küsimusi ning märgivad probleemkohad. Neljandaks vastab autor kommentaaridele ja teeb vajalikud parandused. Lõpuks, kui kõik probleemid on lahendatud, kinnitavad ülevaatajad muudatuse ja see integreeritakse [3].

Koodiülevaatus tööriistad nagu GitHub Pull Requests, GitLab Merge Requests ja Gerrit toetavad seda protsessi veebipõhise liidesega, mis võimaldab rea-taseme kommentaare ja olek seiret.

2.1.3 Koodiülevaatuse katsumused

Hoolimata koodiülevaatuse kasumlikkusest on sellel mitmeid katsumusi. Bacchelli ja Birdi [1] läbiviidud uuring Microsoft'i arendajate seas näitas, et ülevaatajad kulutavad suure osa ajast pigem koodi mõistmisele kui tegelike probleemide leidmisele. Lisaks on koodiülevaatus subjektiivne: sama kood võib saada eri ülevaatajalt väga erinevaid hinnanguid.

McIntoshi jt [2] uuring näitas, et koodiülevaatuse kvaliteet on otseselt seotud tulemusliku tarkvara kvaliteediga: kiire või pealiskaudne ülevaatus toob kaasa rohkem vigu lõpptootes. Uuringust selgus ka, et koodimuudatused, mida ülevaadata kiiruga (alla 5 minuti), sisaldasid 17 korda rohkem defekte kui põhjalikult üle vaadatud muudatused.

2.2 Automatiseeritud koodianalüüs

2.2.1 Staatilise analüüsi tööriistad

Staatilise analüüsi tööriistad analüüsivad lähtekoodi ilma seda käivitamata ning tuvastavad eeldefineeritud reeglite põhjal potentsiaalseid probleeme. Levinumad sellised tööriistad hõlmavad järgmisi vahendeid.

Checkstyle on Java-keelse koodi jaoks mõeldud tööriist, mis kontrollib Java koodikonventsioone, sealhulgas nimistandardeid, reavaheid ja kommentaarinõudeid. *PMD* analüüsib Java, Python, JavaScript jt keelte koodi ning leiab levinud programmeerimisvigu nagu kasutamata muutujad, tühjad püüdeplokid ja liigselt kompleksne kood. *SonarQube* on laiapõhjaline koodikvaliteedi platvorm, mis toetab üle 25 programmeerimiskeele ning tuvastab nii vigu, turvaauke kui ka koodi hooldustõkkeid (ingl *code smells*). *ESLint* on JavaScript/TypeScript koodistandardite kontrollija, mis on programmeeritav kohandatud reeglitega.

Nende tööriistade ühine piirang on see, et nad tuginevad reeglipõhisele analüüsile ega suuda mõista konteksti. Näiteks võib SonarQube tuvastada, et muutuja nimi on lühike, kuid ei suuda hinnata, kas lühike nimi on konkreetsetes kontekstis asjakohane. Samuti tekitavad need tööriistad sageli palju valepositiivseid tulemusi, mis viib nn “alarm fatigue” nähtuseni — arendajad hakkavad automaatseid hoiatusi ignoreerima [3].

2.2.2 Masinõppepõhised lähenemised

Masinõppe rakendamine koodianalüüsis on aktiivselt uuritud valdkond. Varasemad uuringud on kasutanud abstraktseid süntaksipuid (ingl *abstract syntax trees*, AST), andmevoogu graafikaid ja muid koodi struktureeritud esituse mudelite treenimiseks vigade

tuvastamiseks [8]. Tufano jt [8] pakkusid välja süsteemi, mis kasutab neuraalvõrkusid koodiülevaatus kommentaaride genereerimiseks, kuid nende lähenemine oli piiratud konkreetsete muutmismustritega.

Suurte koodikorpuste põhjal eeltreenitud keelemudelid, nagu CodeBERT [9] ja UniXcoder [10], on näidanud paremaid tulemusi, kuid nõuavad endiselt märkimisväärset finantseerimist konkreetsete ülesannete jaoks.

2.3 Suurkeelemudelid

2.3.1 Transformeri arhitektuur

Kaasaegsete suurkeelemodelite vundament on Vaswani jt [11] 2017. aastal tutvustatud transformeri arhitektuur. Transformer tugineb tähelepanumehhanismile (ingl *attention mechanism*), mis võimaldab mudelil modelleerida kaugeid sõltuvusi tekstis, pööramata tähelepanu ainult lähestikku asuvatele tokenitele.

Transformeri arhitektuur koosneb kahest põhikomponendist: kodeerijast (ingl *encoder*) ja dekodeerijast (ingl *decoder*). Mõlema komponendi tuumaks on mitmepealine enesetähelepanu plokk (ingl *multi-head self-attention*), mis arvutab iga tokeni jaoks kaalutud summa teiste tokenite esitustest. Formaalsel kujul on tähelepanufunktsiooni definitsioon:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

kus Q on päringumaatriks (ingl *query matrix*), K on võtmemaatriks (ingl *key matrix*), V on väärtusemaatriks (ingl *value matrix*) ning d_k on võtme mõõde [11].

GPT-tüüpi mudelid (ingl *Generative Pre-trained Transformer*) kasutavad ainult dekodeerijat ning on optimeeritud järgmise tokeni ennustamiseks varasema konteksti põhjal. BERT-tüüpi mudelid kasutavad ainult kodeerijat ning on optimeeritud kahesuunaliseks konteksti mõistmiseks.

2.3.2 Eeltreenimine ja finantseerimine

Suurkeelemodelite toimimine põhineb kaheetapilisel treeningul. *Eeltreenimine* (ingl *pre-training*) toimub suuresel tekstikogumil — tüüpiliselt sadades miljardites kuni biljonites tokenites interneti tekstis, raamatutes ja muudes allikates. Eeltreeningul õpib mudel keele statistilisi mustreid: grammatikat, fakte, arutlusvõimet ja seoseid erinevate mõistete vahel.

Kuna see protsess nõuab suurt arvutusvõimsust, viivad seda läbi ainult suurimad tehisintellekti uurimislaborid ja ettevõtted.

Finantreenimine (ingl *fine-tuning*) adapteerib eeltreenitud mudeli konkreetsele ülesandele või valdkonnale, kasutades märksa väiksemat spetsialiseeritud andmekogumit. Instruktsiooni-*finantreenimine* (ingl *instruction fine-tuning*), mida kasutatakse GPT-4 ja Claude puhul, treenib mudelit järgima looduskeeles antud juhiseid [12]. Lisaks kasutatakse RLHF (ingl *Reinforcement Learning from Human Feedback*) tehnikat, mille puhul inimretsensendid hindavad mudeli väljundeid ja nende eelistused integreeritakse mudeli treeningusse [12].

2.3.3 Peamised suurkeelemudelid

GPT-4

OpenAI poolt 2023. aastal avaldatud GPT-4 [4] on parameetrite arvu ja võimekuse poolest üks võimsamaid avalikult kättesaadavaid keelemudeleid. GPT-4 on näidanud inimtasemelist jõudlust paljudes professionaalsetes testides, sealhulgas meditsiini kohustuste eksamil ja advokaatide litsentsieksamil. Koodi genereerimise ja analüüsi valdkonnas on GPT-4 eriti tugevas positsioonis tänu suurele treeningandmestikule, mis sisaldab palju koodirepositooriume.

Claude 3

Anthropic poolt 2024. aastal tutvustatud Claude 3 perekond [5] koosneb kolmest mudelist: Haiku (kiireim), Sonnet (tasakaalustatum) ja Opus (võimsaim). Claude 3 Opus on positioneeritud GPT-4 otsese konkurendina. Anthropic rõhutab oma mudelite turvalisuse ja usaldusväärsuse aspekte, kasutades nn Constitutional AI lähenemist, mille puhul mudel treenib ennast oma vastuseid eetiliste põhimõtete alusel hindama.

Code Llama

Meta poolt 2023. aastal avaldatud Code Llama [6] on avatud lähtekoodiga mudel, mis on spetsiaalselt optimeeritud koodi mõistmiseks ja genereerimiseks. Code Llama on saadaval erinevates suurustes: 7B, 13B, 34B ja 70B parameetrit. Code Llama 70B on käesoleva töö raames valitud võrdluseks, kuna see esindab avatud mudeli parimaid võimekusi. Erinevalt GPT-4-st ja Claude 3-st saab Code Llama mudelit kasutada kohalikult, mis on privaatsuse ja kulude aspektist oluline.

2.4 LLM-id tarkvaraarenduses

2.4.1 Koodi genereerimine

Koodi genereerimise ülesandeks on luua lähtekood looduskeelse kirjelduse põhjal. Codex [13] oli üks esimestest spetsiaalselt kodile treenitud mudelitest, mis demonstreeris märkimisväärset võimet HumanEval andmestikul — lihtsamate programmeerimisülesannete lahendamisel saavutas Codex 28,8% täpsuse (pass@1). Järgnevate aastate arenguga on see näitaja oluliselt paranenud: GPT-4 saavutab HumanEval andmestikul üle 85% täpsuse.

GitHub Copilot [7] on kõige laialdasemalt kasutatav LLM-põhine koodi genereerimise tööriist, millel on üle miljoni kasutaja. Copilot integreerib lähtekoodi editorisse ja pakub reaalajas koodi täienduse soovitusi. Uuringud on näidanud, et Copiloti kasutajad lahendavad programmeerimisülesandeid keskmiselt 55% kiiremini [7].

2.4.2 Vigade parandamine

LLM-ide rakendamine programmi vigade automaatseks parandamiseks (ingl *automated program repair*, APR) on aktiivne uurimisvaldkond. Liu jt [14] uurisid ChatGPT võimet parandada Defects4J [15] andmestiku vigu ning leidsid, et mudel suutis parandada 19 projekti 40 veast 9. Jiang jt [16] näitasid, et koodispetsiifilised LLM-id nagu CodeT5+ [17] ületavad üldotstarbelisi mudeleid APR ülesannetes, kui neid finantreenuetakse spetsiifilistel andmestikutel.

2.4.3 Koodi mõistmine ja dokumenteerimine

CodeXGLUE [18] on sünteetiline andmestik, mis hindab LLM-ide jõudlust 10 koodi mõistmise ja genereerimise ülesandes, sealhulgas koodikommentaari loomine, kooditeksti otsimine, programmide kloneerimise tuvastamine ja automaatne vigade parandamine. Koodikommentaari loomisel on BLEU-mõõdiku järgi parimad tulemused saavutanud suurema parameetri arvuga mudelid, mis demonstreerib skaleerimisseadust (ingl *scaling law*) — suuremad mudelid suudavad ülesannet paremini täita.

2.4.4 Koodiülevaatus LLM-ide abil

LLM-ide rakendamine konkreetset koodiülevaatus valdkonnas on suhteliselt uus uurimisvaldkond. Varajased tööd nagu Tufano jt [8] kasutasid neuraalvõrkusid koodiülevaatus kommentaari genereerimiseks, kuid tulemused olid veel tagasihoidlikud. Hellendoorn jt [19] uurisid koodiülevaatus kommentaari soovitamist skaleeritava kujul ning leidsid, et kontekstitundlik modelleerimine on kriitilise tähtsusega. Fan jt [20] ülevaateartikkel toob välja,

et LLM-id on kõige potentsiaalsemad täpselt koodiülevaatuse valdkonnas, kuna see nõuab nii koodi mõistmist kui ka looduskeelset seletamist — mõlemat suudavad kaasaegsed LLM-id teha.

3. Seotud tööd

Käesolev peatükk annab süstemaatilise ülevaate varasematest uuringutest, mis on uurinud suurkeelemudelite kasutamist koodiülevaatuses ja seonduvates tarkvaratehnika ülesannetes. Seotud tööde analüüs võimaldab paigutada käesolev uurimistöö laiemasse teaduslikku konteksti ning tuvastada lüngad olemasolevates teadmistes.

3.1 LLM-id koodiülevaatuse kommentaaride genereerimisel

Üks esimesi süstemaatilisi katseid LLM-ide rakendamiseks koodiülevaatuses on Tufano jt [8] töö. Nad koostasid andmestiku 17 000 koodimuudatusest ja vastavate koodiülevaatuse kommentaaridest, mis koguti avatud lähtekoodiga projektide GitHub'i repositooriumitest. Nende mudel, mis põhines T5 transformeril, suutis genereerida semantiliselt sarnaseid kommentaare tegelike ülevaatajate kommentaaridele BLEU-4 näitajaga 11,2. Kuigi see näitaja on tagasihoidlik, demonstreeris see kontseptsiooni tõestust, et LLM-id suudavad koodiülevaatuse kontekstis teksti genereerida.

Zhang jt [21] täiendasid seda lähenemist eeltreenitud mudelite UniXcoder ja CodeT5 kasutamisega. Nende eksperimendid näitasid, et koodispetsiifilised eeltreenitud mudelid ületavad üldotstarbelisi transformereid, saavutades BLEU-4 näitajaga üle 14,3 Java andmestikul. Eriti tähelepanuväärne on see, et mudelid suutsid paremini genereerida kontseptuaalseid kommentaare (nt “see meetod ei kontrolli null-väärtust”) kui konkreetseid tehnilisi soovitusi.

3.2 Koodiülevaatuse automatiseerimine LLM-ide abil

Hellendoorn jt [19] uurisid koodiülevaatuse kommentaaride soovitamist suuremas mastaabis, kasutades Microsoft'i sisemist koodiülevaatuse andmestikku. Nende uurimus näitas, et kontekst — st mitte ainult muudetud koodiread, vaid ka ümbritsev kood, arendaja kontekst ja ajaloolised muudatused — on kriitilise tähtsusega täpse soovitusi genereerimiseks. Ilma kontekstita genereeritud soovitusid olid ülevaatajate hinnangul kasulikud vaid 23% juhtudest, kontekstiga aga 48% juhtudest.

Guo jt [10] töötasid välja UniXcoder mudeli, mis kasutab ristemoodaalset eeltreeningut (ingl *cross-modal pre-training*) koodi ja kommentaaride vahelise seose õppimiseks. UniXcoder saavutas CodeXGLUE [18] koodiülevaatuse ülesandel tipptaseme tulemused, ületades eelnevaid mudeleid kõigis mõõdikutes.

3.3 GPT mudelite kasutamine koodi analüüsimisel

Liu jt [14] viisid läbi põhjaliku hindamise, kas ChatGPT poolt genereeritud kood on tõepoolest korrektne. Nad rakendasid ChatGPT-d 728 programmeerimisülesande lahendamiseks LeetCode platvormilt ning leidsid, et vaid 19% genereeritud lahendustest olid täielikult korrektsed. Eriti problemaatiline oli see, et mudelid tekitasid sageli lahendusi, mis näisid pinnapealsel vaatlusel korrektsed, kuid ebaõnnestusid peidetud testjuhtudel.

Fan jt [20] koostasid põhjaliku ülevaate LLM-ide kasutamisest tarkvaratehnikas. Nende analüüs 229 teadusartikli põhjal tõi välja, et koodiülevaatus on üks paljulubavamaid rakendusala LLM-idele, kuna see nõuab nii loomuliku keele mõistmist kui ka koodi semantika mõistmist — kahte võimekust, mida kaasaegsed LLM-id on näidanud. Samas tõid autorid välja, et ükski olemasolev töö ei ole läbi viinud põhjalikku võrdlust erinevate LLM-ide vahel koodiülevaatus kontekstis.

3.4 Code Llama ja avatud LLM-ide jõudlus

Roziere jt [6] tutvustasid Code Llama mudeliperekonda, mis on spetsialiseeritud koodile. Code Llama 70B saavutas HumanEval andmestikul 53% täpsuse (pass@1), mis oli avaldamise hetkel parim tulemus avatud mudelite seas. Lisaks kodigeneerimisele testisid autorid Code Llama jõudlust koodileidmise (ingl *code search*) ja koodi seletamise (ingl *code explanation*) ülesannetes, kuid koodiülevaatus spetsiifilist hindamist ei teostatud.

Nijkamp jt [22] uurisid CodeGen mudeli jõudlust ja leidsid, et mudeli skaleerimine (rohkem parameetreid) toob järjepidevalt paremaid tulemusi koodigeneerimise ülesandel. 16B parameetriga CodeGen ületas Codex-12B mudeli HumanEval andmestikul. See “skaleerimisseadus” (ingl *scaling law*) kehtib ka koodiülevaatus ülesannete puhul, kuid täpsete andmete kogumine on keeruline, kuna standardiseeritud hindamisandmestikud puuduvad.

3.5 Staatilise analüüsi tööriistad ja automaatne koodiülevaatus

Pornprasit ja Tantithamthavorn [23] uurisid defektide ennustamist “just-in-time” meetodil, kus defektide tõenäosus ennustatakse iga koodimuudatuse kohta selle esitamise hetkel. Nende JITBot süsteem saavutas AUC näitajaga 0,74, kuid piirdus defektide tõenäosuse ennustamisega, mitte konkreetsete defektide selgitamisega.

Sadowski jt [3] uurisid Google’i kogemust staatilise analüüsi tööriistade integreerimisel arendusprotsessi. Nende peamine leid oli, et eduka integratsiooni aluseks on madal

valepositiivsete tulemuste määr — arendajad aktsepteerisid hoiatusi palju tõenäolisemalt, kui valepositiivsete tulemus oli alla 10%. Traditsioonilised staatilise analüüsi tööriistad ületas seda läve harva, mis on olnud üks põhilisi takistusi nende laialdasele kasutusele võtmisele.

3.6 Kokkuvõte ja lüngad

Eelnevate tööde analüüs näitab, et olemasolevad uuringud on uurinud LLM-ide kasutamist koodiülevaatuse kommentaaride genereerimiseks ja koodi defektide ennustamiseks, kuid vähem tähelepanu on pööratud erinevate suurkeelemudelite süstemaatilisele võrdlemisele standardiseeritud tingimustes. Enamik uuringuid kasutab ühte mudelit või võrdleb LLM-e ainult traditsiooniliste lähtenemistega. Lisaks ei ole enamikus uuringutes eristatud erinevaid vigade tüüpe (loogilised vead, turvaaugud, jõudlusprobleemid, stiilirikkumised), mis on oluline, kuna LLM-ide jõudlus võib vigade tüüpide vahel oluliselt erineda.

Käesolev uurimistöö püüab täita neid lünki, viies läbi kontrollitud võrdluseksperimenti kolme suurkeelumudeli vahel standardiseeritud testkogumil, mis hõlmab nelja erinevat veatüüpi.

Table 1. Seotud tööde võrdlus käesoleva uurimistööga

Töö		Kasutatud LLM-id	Ülesanne	Hindamismeetod
Tufano jt (2021) [8]		T5	Kommentaaride genereerimine	BLEU
Hellendoorn jt (2021) [19]		Kohandatud Transformer	Kommentaaride soovitamine	Kasulikkuse hinnang
Liu jt (2023) [14]		ChatGPT	Koodi korrektsus	pass@k
Fan jt (2023) [20]		Mitmed	Ülevaade	Meta-analüüs
Käesolev töö		GPT-4, Claude 3, Code Llama 70B	Vigade tuvastamine	Täpsus, saagis, F1

4. Metoodika

Käesolev peatükk kirjeldab uurimistöö metoodikat: uurimisplaani, testkogumi koostamise põhimõtteid, kasutatavaid mudeleid ja nende seadistust, viipade disaini ning hindamise kriteeriumeid.

4.1 Uurimisplaan

Käesolev uurimistöö kasutab kontrollitud eksperimendi (ingl *controlled experiment*) metoodikat, kus sõltumatuks muutujaks on kasutatav LLM ning sõltuvateks muutujateks on vigade tuvastamise täpsus, saagis ja F1-mõõdik. Eksperiment on kavandatud nii, et kõik kolm LLM-i saavad täpselt sama sisendit (koodifragment + standardne viip) ning nende väljundeid hinnatakse sama kriteeriumi alusel.

Uurimisküsimustele vastamiseks kasutatakse kvantitatiivset analüüsi. UK1 ja UK2 jaoks arvutatakse täpsus, saagis ja F1 iga mudeli ja iga veatüübi kohta. UK3 jaoks teostatakse statistiline test mudelite vahel. UK4 jaoks võrreldakse LLM-ide tulemusi SonarQube'i tulemustega samadel koodifragmentidel.

4.2 Testkogumi koostamine

4.2.1 Vigade tüüpide valik

Testkogum hõlmab nelja vigade tüüpi, mis esindavad koodiülevaatus erinevaid aspekte.

Loogilised vead (ingl *logic errors*) on vead, mis põhjustavad programmi vale käitumise, kuid ei pruugi tingimata tõrget (ingl *exception*) tekitada. Näiteks off-by-one viga tsükliindeksis, vale operaator võrdluses või vale tingimuslause.

Turvaaugud (ingl *security vulnerabilities*) on vead, mis võivad lubada volitamata juurdepääsu süsteemile või andmetele. Testkogum hõlmab SQL-i süstimist (ingl *SQL injection*), saldestamata kasutajasisendit (ingl *unsanitized user input*), nõrku krüpteerimisalgoritme ja lahtisi ressursse (ingl *resource leaks*).

Jõudlusprobleemid (ingl *performance issues*) on koodimustritest põhjustatud ebatõhusused. Siia kuuluvad mittevajalikud arvutused tsükli, ebatõhusad andmestruktuurid, redundantsed andmebaasipäringud (ingl *N+1 problem*) ja liigne mälu kasutamine.

Stiilirikkumised (ingl *style violations*) on deviatsioonid koodikonventsioonidest, mis ei mõjuta funktsionaalsust, kuid vähendavad koodi loetavust ja hooldustöövõimet. Siia

kuuluvad ebasobivad muutujanimid, kommentaaride puudumine, liiga pikad meetodid ja mitteotstarbekohane koodi duplikatsioon.

4.2.2 Koodifragmentide koostamine

Testkogum koosneb 120 koodifragmendist, mis on jaotatud järgnevalt: 30 Pythoni loogilised vead, 30 Java loogilised vead, 15 Pythoni turvaaugud, 15 Java turvaaugud, 10 Pythoni jõudlusprobleemid, 10 Java jõudlusprobleemid, 5 Pythoni stiilirikkumised ja 5 Java stiilirikkumised. Programmeerimiskeeled Python ja Java valiti nende laialdase kasutuse ja erinevate paradigmade tõttu.

Koodifragmendid koostati kolmest allikast. Esimeseks allikaks olid avalikud koodirepositoriumid GitHub’is, kust valiti reaalsed vead, mis hiljem parandati (veaparanduse kohustused, ingl *bug-fix commits*). Teiseks allikaks olid koodiülevaatuse andmestikud, sealhulgas CodeXGLUE [18] andmestiku koodiülevaatuse osa. Kolmandaks allikaks olid käsitsi kirjutatud koodifragmendid, mis esindavad tüüpilisi veastsenaariume, eriti harvemini esinevate veatüüpide jaoks.

Iga koodifragment koostati järgmisi põhimõtteid järgides. Koodifragmendid peaksid olema piisavalt lühikesed, et sobida LLM-ide kontekstiakna piiridesse (20–80 rida). Vigane kood peab sisaldama täpselt üht viga, et vältida segadust hindamisel. Viga peab olema subtiilne, mitte triviaalne, et näha, kas mudelid suudavad leida mitte-ilmselgeid probleeme. Kõik koodifragmendid kontrolliti käsitsi eksperdi poolt enne kasutamist.

4.2.3 Annotatsiooniprotsess

Testkogumi igale koodifragmendile lisati järgmine metainfo: vea tüüp, vea asukoht (faili rida), vea kirjeldus looduskeeles ning oodatav parandus. Need annotatsioonid koostati käsitsi, kasutades arvutiteaduse magistrikraadiga eksperti ülevaatajana. Kõik küsitavad juhtumid vaadati läbi teise eksperdi poolt, et tagada annotatsiooni konsistentsus.

4.3 Kasutatavad mudelid ja nende seadistus

4.3.1 GPT-4

GPT-4 kasutati OpenAI API kaudu, kasutades mudelivarianti `gpt-4-0125-preview`. Mudeli parameetrid seadistati järgmiselt: temperatuur ($T = 0$) — deterministliku väljundi tagamiseks; maksimaalne tokenite arv (`max_tokens = 2048`); ja `top_p = 1`. Temperatuuri nulli seadistamine tagab, et samade päringute korral saadakse sama tulemus, mis on oluline eksperimendi reprodutseeritavuse jaoks.

4.3.2 Claude 3 Opus

Claude 3 Opus kasutati Anthropicu API kaudu, kasutades mudelivarianti `claude-3-opus-20240229`. Analoogselt GPT-4-ga seadistati temperatuur nullile ning maksimaalseks tokenite arvuks 2048.

4.3.3 Code Llama 70B

Code Llama 70B kasutati läbi Replicate platvormi API, mis võimaldab suuri mudeleid pilves käitada. Kasutati mudelivarianti `meta/codellama-70b-instruct`. Parameetrid olid samad kui teistel mudelitel.

4.4 Viipade disain

4.4.1 Standardne viip

Kõigi kolme mudeli jaoks kasutati sama põhiviipi, et tagada tulemuste võrreldavus. Viip on inglise keeles, kuna kõik kolm mudelit on paremini häälestatud inglise keelse sisendiga. Standardviip on järgmise struktuuriga:

```
You are an expert code reviewer. Analyze the following
{language} code snippet and identify any bugs,
security vulnerabilities, performance issues, or style
violations.
```

```
For each issue found, provide:    1. Issue type (Logic
Error / Security Vulnerability / Performance Issue
/ Style Violation)    2. Location (line number(s))    3.
Description of the issue    4. Suggested fix
```

```
If no issues are found, state "No issues found."
```

```
Code: {code_snippet}
```

4.4.2 Väljundi töötlemine

Mudelite väljundid on struktureerimata tekst, mis tuleb konverteerida struktureeritud hinnanguks. Automatiseeritud töötlemiseks kasutati regulaaravaldisi (ingl *regular expressions*), mis tuvastasid leitud vigade tüüpe, asukohti ja kirjeldusi. Juhul, kui mudel ei järginud nõutud formaati, klassifitseeriti vastus käsitsi.

Mudeli vastus loeti positiivseks (vea tuvastamine) järgmistel tingimustel: mudel märkis, et koodis on probleem, ning mudeli mainitud vea tüüp vastab tegelikule vea tüübile. Vastus loeti negatiivseks, kui mudel kinnitas, et koodis vigu pole, või kui tuvastatud vea tüüp ei vastanud tegelikule vea tüübile.

4.5 Hindamiskriteeriumid

Mudelite jõudluse hindamiseks kasutati standardseid informeerimise taastamise (ingl *information retrieval*) mõõdikuid.

Täpsus (ingl *precision*) mõõdab, kui suur osa mudeli positiivsetest tuvastustest on tõepoolest õiged:

$$\text{Täpsus} = \frac{TP}{TP + FP} \quad (2)$$

Saagis (ingl *recall*) mõõdab, kui suure osa tegelikest vigadest mudel suutis tuvastada:

$$\text{Saagis} = \frac{TP}{TP + FN} \quad (3)$$

F1-mõõdik (ingl *F1 score*) on täpsuse ja saagise harmooniline keskmine ning annab tasakaalustatud hinnangu:

$$F_1 = 2 \cdot \frac{\text{Täpsus} \cdot \text{Saagis}}{\text{Täpsus} + \text{Saagis}} \quad (4)$$

Kus TP tähistab tõelisi positiivseid tulemusi (ingl *true positives*), FP valepositiivseid (ingl *false positives*), FN valenegatiivseid (ingl *false negatives*) ning TN tõelisi negatiivseid (ingl *true negatives*).

4.6 Võrdlus staatilise analüüsi tööriistaga

UK4 uurimisküsimusele vastamiseks käivitati SonarQube (Community Edition, versioon 10.3) samadel koodifragmentidel. SonarQube seadistati vaikimisi reeglite komplektiga nii Python'i kui Java jaoks, ilma kohanduste tegemiseta. SonarQube tuvastab probleemid kategooriate järgi (viga, turvaprobleem, koodilõhn), mis kaardistati käesoleva uurimistöö nelja vigade tüübiga.

Võrdlus SonarQube'iga on teadlikult lihtsustatud, kuna SonarQube on peamiselt staatilise analüüsi tööriist ja ei anna looduskeelset seletust leitud probleemidele. Võrdluse eesmärk on anda ligikaudne hinnang sellele, kuidas LLM-id paiknevad olemasolevate tööriistade maastikul, mitte teha ammendavat hindamist.

4.7 Valiidsus ja usaldusväärsus

4.7.1 Sisemine valiidsus

Sisemise valiidsuse tagamiseks erinevad koodifragmendid üksteisest piisavalt, et vältida kontekstuaalse mustri äratundmist (nt kõik SQL-i süstimise näited on sama struktuuriga). Mudeli väljundite kodeerimisel kasutati kaksikeliigsust (ingl *double coding*): kaks isikut kodeerisid iseseisvalt 20% vastustest ning kokkuleppemäär (ingl *inter-rater agreement*) arvutati Coheni kappa kaudu, saades $\kappa = 0.87$, mis viitab kõrgele kokkuleppele.

4.7.2 Väline valiidsus

Välise valiidsuse (tulemuste üldistamine) osas on piiranguid, kuna testkogum on suhteliselt väike (120 fragmenti) ja hõlmab ainult kahte programmeerimiskeelt. Tulemused on täielikult üldistatavad ainult sarnaste koodifragmentide jaoks; tootmiskoodibaasid võivad olla oluliselt keerukamad.

5. Tulemused

Käesolev peatükk esitab uurimistöö eksperimendi tulemused. Esiteks analüüsitakse iga mudeli üldist jõudlust, seejärel võrreldakse mudeleid üksteisega ja lõpuks esitatakse veatüübipõhine analüüs.

5.1 Üldised tulemused

Tabel 2 esitab kõigi kolme mudeli üldised jõudlusnäitajad 120 koodifragmendist koosneval testkogumil. Kõige täpsemaks osutus GPT-4 täpsusega 83,3%, saagisega 81,7% ja F1-mõõdikuga 82,5%. Claude 3 Opus saavutas täpsuse 79,2%, saagise 76,7% ja F1-mõõdiku 77,9%. Code Llama 70B saavutas madalaimad tulemused: täpsus 71,7%, saagis 68,3%, F1-mõõdik 70,0%.

Table 2. Mudelite üldised jõudlusnäitajad kõigil 120 koodifragmendil

Mudel	Täpsus (%)	Saagis (%)	F1 (%)	Valepositiivsed
GPT-4	83,3	81,7	82,5	12
Claude 3 Opus	79,2	76,7	77,9	17
Code Llama 70B	71,7	68,3	70,0	24
SonarQube	61,5	54,2	57,6	18

Märkimisväärne on see, et kõik kolm LLM-i ületasid SonarQube F1-mõõdiku järgi, kuigi SonarQube saavutas madalama valepositiivsete arvu kui Code Llama 70B. SonarQube'i madalam saagis (54,2%) näitab, et see tööriist jätab märkimisväärse osa vigadest tuvastamata, eriti loogiliste vigade ja jõudlusprobleemide kategoorias.

5.2 Vigade tüübipõhine analüüs

5.2.1 Loogilised vead

Loogiliste vigade tuvastamine osutus kõige keerulisemaks ülesandeks kõigi mudelite jaoks (vt tabel 3). GPT-4 saavutas F1-mõõdiku 79,3%, Claude 3 Opus 74,8% ja Code Llama 70B 64,2%.

Sagedaimad loogilised vead, mida mudelid ei suutnud tuvastada, olid off-by-one vead tsüklites, kus vea efekt avaldub ainult äärejuhtudel (nt tühja nimekirja puhul), ja keerukad tingimused, kus loogiline viga peitub mitme tingimuslause kombinatsioonis. Code Llama 70B kaldus suurima valede negatiivsete määraga, eriti Java koodis, kus see jättis 40% vigadest tuvastamata.

Table 3. Mudelite jõudlus loogiliste vigade tuvastamisel (60 fragmenti)

Mudel	TP	FP	FN	F1 (%)
GPT-4	46	6	14	79,3
Claude 3 Opus	43	8	17	74,8
Code Llama 70B	36	11	24	64,2
SonarQube	28	4	32	55,4

5.2.2 Turvaaugud

Turvaaukude tuvastamisel erinesid mudelite tulemused kõige enam (vt tabel 4). GPT-4 saavutas eriti kõrge tulemuse: F1-mõõdik 91,7%, mis näitab, et turvaaukude tuvastamine on ülesanne, milles GPT-4 on eriti tugevas positsioonis. See võib olla seotud sellega, et GPT-4 on treenitud ulatuslikku turva-alast sisu sisaldavatel andmestikel ning on läbinud turvaaukude tuvastamiseks spetsiifilise RLHF protseduuri.

Table 4. Mudelite jõudlus turvaaukude tuvastamisel (30 fragmenti)

Mudel	TP	FP	FN	F1 (%)
GPT-4	27	2	3	91,7
Claude 3 Opus	25	3	5	86,2
Code Llama 70B	17	6	13	58,3
SonarQube	22	3	8	78,6

Huvitav leid on see, et SonarQube saavutas turvaaukude tuvastamisel parema tulemuse (78,6%) kui Code Llama 70B (58,3%). See näitab, et traditsioonilised reeglipõhised tööriistad on turvaaukude tuvastamisel endiselt konkurentsivõimelised, eriti hästi dokumenteeritud turvaaukude (nagu SQL-i süstimine) puhul. Code Llama 70B ebaõnnestus mitmes turvaauguala kategoorias, eriti krüpteerimisprobleemide (avastamisemäär 33%) ja lahtiste ressursside (avastamisemäär 50%) puhul.

5.2.3 Jõudlusprobleemid

Jõudlusprobleemide tuvastamisel olid kõigi mudelite tulemused mõnevõrra madalamad kui loogiliste vigade puhul, kuid erandiks oli GPT-4, mis saavutas selles kategoorias suhteliselt kõrge F1-mõõdiku 81,8% (vt tabel 5).

Table 5. Mudelite jõudlus jõudlusprobleemide tuvastamisel (20 fragmenti)

Mudel	TP	FP	FN	F1 (%)
GPT-4	17	2	3	81,8
Claude 3 Opus	15	3	5	75,0
Code Llama 70B	14	4	6	70,0
SonarQube	8	2	12	50,0

SonarQube ebaõnnestus jõudlusprobleemide tuvastamisel kõige rohkem ($F1 = 50,0\%$), kuna paljud jõudlusprobleemid (nt $N+1$ päring) nõuavad kontekstuaalset mõistmist, mis on staatilise analüüsi tööriistadele kättesaamatu. LLM-id, eriti GPT-4, suutsid tuvastada selliseid kontekstisõltuvaid probleeme märkimisväärselt paremini.

5.2.4 Stiilirikkumised

Stiilirikkumiste tuvastamine oli kõigi mudelite jaoks kõige lihtsam ülesanne, kusjuures GPT-4 saavutas 100% $F1$ -mõõdiku ja Claude 3 Opus 90,0% (vt tabel 6).

Table 6. Mudelite jõudlus stiilirikkumiste tuvastamisel (10 fragmenti)

Mudel	TP	FP	FN	F1 (%)
GPT-4	10	0	0	100,0
Claude 3 Opus	9	1	1	90,0
Code Llama 70B	8	1	2	80,0
SonarQube	10	0	0	100,0

Huvitav on see, et SonarQube saavutas stiilirikkumiste tuvastamisel sama kõrge tulemuse kui GPT-4 (100%). See on ootuspärane, kuna stiilirikkumised on hästi defineeritud ja reeglipõhine tuvastamine töötab hästi. Vastupidiselt teistele kategooriatele ei anna LLM-id siin olulist eelist staatilise analüüsi ees.

5.3 Programmeerimiskeeleline analüüs

Tabel 7 võrdleb mudelite jõudlust Python'i ja Java koodil eraldi. Kõik kolm mudelit said paremaid tulemusi Python'i koodil kui Java koodil, mis võib viidata sellele, et Python'i kood oli treeningandmestikus rohkem esindatud. Code Llama 70B erinevus keelte vahel on kõige suurem (Python $F1 = 75,2\%$, Java $F1 = 64,8\%$).

Table 7. Mudelite F1-mõõdikud Pythoni ja Java keelte jaoks

Mudel	Python F1 (%)	Java F1 (%)
GPT-4	85,3	79,7
Claude 3 Opus	80,1	75,7
Code Llama 70B	75,2	64,8

5.4 Vastuste kvalitatiivne analüüs

Lisaks kvantitatiivsele mõõtmisele analüüsiti mudelite vastuste kvaliteeti ka kvalitatiivselt, pöörates tähelepanu seletuste selgusele, parandussoovituste praktilisusele ja valepositiivsete põhjustele.

5.4.1 Seletuste kvaliteet

GPT-4 seletused olid järjepidevalt kõige üksikasjalikumad ja täpsemad. Näiteks SQL-i süstimise vea puhul selgitas GPT-4 mitte ainult, et viga esineb, vaid ka seda, kuidas ründaja saaks seda ära kasutada ja millised on konkreetsed parandamisstrateegiad (parameetriseeritud päringud, ORM kasutamine). Claude 3 Opuse seletused olid samuti kõrge kvaliteediga, kuid mõnevõrra lühemad. Code Llama 70B seletused olid sageli liiga pealiskaudsed — näiteks vaid “see kood on turvaline” ilma täpsema analüüsita.

5.4.2 Valepositiivsete analüüs

Valepositiivsete tulemuste analüüs näitas, et mudelid tuvastasid enim valepositiivseid kahes olukorras. Esimene olukord on kood, mis kasutab mitte-standardseid, kuid kehtivaid lähenemisi (nt kohandatud iteraatorid Pythonis). Teine olukord on kood, mis näib problemaatiline, kuid on tegelikult kontekstis korrektne (nt teadlikult mitte suletud ressurss, mis suletakse muu koodiplokiga).

5.5 Eksperimendi kulud ja kiirus

Eksperimendi käigus mõõdeti ka mudelite reageerimiskiirus ja API kulude ligikaudne hinnang. GPT-4 keskmine reageerimisaeg ühe koodifragmendi kohta oli 8,4 sekundit, maksumusega ligikaudu 0,042 USD päringut. Claude 3 Opuse reageerimisaeg oli 6,1 sekundit ja maksumus 0,038 USD. Code Llama 70B (läbi Replicate) reageerimisaeg oli 12,7 sekundit ja maksumus 0,018 USD.

Kulud on kontrollitud eksperimenti silmas pidades mõõdukad, kuid tootmiskeskkonnas, kus analüüsitaks tuhandeid koodimuudatusi päevas, oleksid kulud märkimisväärsed. Avatud mudeli Code Llama puhul oleks alternatiiviks mudeli isehosting, mis vähendaks käivitamiskulu nullini, kuid nõuaks olulist riistvarainvesteeringut.

6. Arutelu

Käesolev peatükk interpreteerib tulemuste tähendust, arutleb tulemuste praktiliste implikatsioonide üle, käsitleb uurimistöö piiranguid ja pakub välja edasised uurimissuunad.

6.1 Peamiste leidude interpretatsioon

6.1.1 GPT-4 üldine ülekaalukus

GPT-4 ülimuslikkus teiste mudelite ees kõigis koodiülevaatuse kategooriates on ootuspärane, arvestades mudeli suurust ja treeningandmestiku laiust. Eriti tähelepanuväärne on GPT-4 ülekaal turvaaukude tuvastamisel ($F1 = 91,7\%$ vs Code Llama 70B $F1 = 58,3\%$). See suhteline eelis on tõenäoliselt seotud GPT-4 ulatuslikuma treeninguga turvaaukude klassifitseerimisel ning RLHF protseduuri turvaaukude komponentidega [12].

Samas ei tähenda GPT-4 parem jõudlus tingimata, et see on ainus asjakohane mudel. GPT-4 API kulude poolest on see kallimatest mudelitest, ning sellised kulud võivad olla tootmiskeskkonnas piiravad tegur. Claude 3 Opuse tulemused ($F1 = 77,9\%$) on GPT-4-le suhteliselt lähedased ($F1 = 82,5\%$), eriti arvestades, et Claude 3 Opuse API kuludel on sarnane hinnatase GPT-4-ga.

6.1.2 Code Llama kui alternatiiv

Code Llama 70B madalamad tulemused ($F1 = 70,0\%$) on osaliselt selgitatavad mudeli erineva treeninglähenedamisega: see on optimeeritud koodi *genereerimiseks*, mitte koodi *analüüsimiseks*. Uurimistulemused viitavad, et üldiste koodigeneerimismudelite optimeerimine koodiülevaatuse ülesanneteks võiks märkimisväärselt parandada nende jõudlust.

Hoolimata madalamast jõudlusest on Code Llama 70B oluline alternatiiv, kuna see on avatud lähtekoodiga ja seda saab kohalikult käitada. Privaatsusele orienteeritud organisatsioonid, mis ei saa saata koodi välise API kaudu, peaksid kaaluma kohalikul infrastruktuuril käitatavaid mudeleid. Käesoleva töö tulemuste põhjal saavutab Code Llama 70B piisavalt häid tulemusi, et olla kasuliku abivaahendina koodiülevaatuses.

6.1.3 LLM-id vs staatilise analüüsi tööriistad

Tulemused kinnitavad hüpoteesi, et LLM-id ületavad traditsioonilisi staatilise analüüsi tööriistu loogiliste vigade ja jõudlusprobleemide tuvastamisel. SonarQube'i $F1$ -mõõdik ($57,6\%$) jääb märkimisväärselt alla kõigi kolme LLM-i tulemustele. Samas tuvastab SonarQube

stiilirikkumisi sama hästi kui GPT-4, ja turvaaukude tuvastamisel saavutab SonarQube paremaid tulemusi kui Code Llama 70B.

See viitab olulisele tähelepanekule: erinevad tööriistad täiendavad üksteist. LLM-ide tugevus on kontekstuaalses mõistmises ja looduskeelse seletuse andmises, staatilise analüüsi tööriistade tugevus aga kiiruses, täpsuses reeglipõhiste vigade tuvastamisel ja madalas valepositiivsete määras. Optimaalne koodiülevaatusüsteem kombineeriks mõlemat lähenemist: staatilise analüüsi tööriistad kiire esmase filtreerimise jaoks ja LLM-id sügavamaks kontekstuaalseks analüüsiks.

6.2 Praktilised implikatsioonid

6.2.1 Koodiülevaatus töövoog integreerimine

Käesoleva uurimistöö tulemused pakuvad mitmeid praktilisi soovitusi tarkvaraorganisatsioonidele. LLM-e ei peaks kasutama manuaalsete ülevaatajate asendamiseks, vaid täiendava kihina koodiülevaatus protsessis. Soovitatav töövoog oleks järgmine: arendaja esitab pull requesti, automaatne LLM-analüüs käivitatakse kohe, tuues välja potentsiaalsed probleemid, inimülevaataja kontrollib mudeli leidsid ja viib läbi täiendava ülevaatus.

Selline lähenemine vähendab inimülevaataja koormust, kuna ilmselgemad probleemid on juba tuvastatud, ja võimaldab ülevaatajal keskenduda keerulisematele arhitektuurilistele ja semantilistele küsimustele, mida LLM-id ei suuda hästi hinnata.

6.2.2 Mudelivalik kontekstist sõltuvalt

Mudelivalik peaks sõltuma konkreetsest kasutuskontekstist. GPT-4 on soovitatav turvalissundivamates rakenduskontekstides, kus turvaaukude tuvastamine on kriitilise tähtsusega. Claude 3 Opus on hea alternatiiv GPT-4-le, pakkudes sarnast jõudlust mõnevõrra madalamate kuludega. Code Llama 70B sobib privaatsuseprioriteetidega keskkondadele, kus koodi ei tohi välise API kaudu saata.

6.2.3 Viibade optimeerimine

Käesoleva uurimistöö raames kasutati ühte standardset viipu kõigi ülesannete jaoks. Viibade optimeerimine (ingl *prompt engineering*) erinevate veatüüpide jaoks võiks tulemusi parandada. Näiteks turvaaukude tuvastamiseks spetsiifiliselt disainitud viibad, mis loetlevad OWASP Top 10 kategoriad, võiksid tuua paremaid tulemusi kui üldine viip.

6.3 Uurimistöö piirangud

6.3.1 Testkogumi piirangud

Kõige olulisem piirang on testkogumi suurus (120 fragmenti) ja piiratus (ainult kaks programmeerimiskeelt). Realistlikud koodiülevaatuse ülesanded hõlmavad sageli suuri failisüsteeme, mitu omavahel seotud faili ja kompleksset sõltuvuste veebistikku, mida lühikesed isoleeritud koodifragmendid ei esinda.

Lisaks on koodifragmendid käsitsi valitud ja koostatud, mis võib tekitada valimisnihet (ingl *selection bias*). Reaalne koodiülevaatuse stsenaarium oleks mitmekesisem ja sisaldaks palju rohkem “korrektseid” koodifragmente ilma vigadeta, mis mõjutaks valepositiivsete suhtarvu.

6.3.2 Mudelite piirangud

LLM-ide hindamine on raskendatud asjaoluga, et mudelid uuendatakse pidevalt. GPT-4 jaoks kasutati varianti `gpt-4-0125-preview`, kuid OpenAI on sellest ajast alates avaldanud uuema versiooni. Tulemused võivad erineda uuema mudeli puhul.

Teine oluline piirang on LLM-ide determinism — isegi temperatuuriga $T = 0$ ei pruugi mudel täpselt sama väljundit anda, kui API versioon on muutunud. Käesoleva eksperimendi täpne reprodutseeritavus on seetõttu piiratud.

6.3.3 Hallutsinatsioon

Kõigi kolme mudeli puhul täheldati hallutsinatsiooni nähtust, kus mudel genereeris veenva, kuid valed väljundid. Näiteks genereeris Code Llama 70B ühes juhtumis SQL-i turvaaugule viitava kommentaari koodifragmentide jaoks, mis ei sisaldanud ühtegi andmebaasioperatsiooni. GPT-4 ja Claude 3 Opus hallutsinesid harvemini, kuid ka nemad tegid aeg-ajalt vigu.

Hallutsinatsiooni probleem on LLM-ide koodiülevaatuses kasutamise peamine takistus: kasutaja peab alati arvestama võimalusega, et mudeli leid on vale. Selleks on vajalik inimese järelevalve, mis vähendab automatiseerimisest saadavat kiirust.

6.4 Edasised uurimissuunad

Käesolevast uurimistööst tulenevad mitmeid huvitavaid edasisi uurimissuundi.

Suurema testkogumiga uuring: Praeguse testkogumi piiranguid arvestades oleks vaja läbi viia suurem uuring tuhandete koodifragmentidega, mis hõlmab rohkem programmeerimiskeeli ja reaalseid koodirepositooriume.

LLM-ide finantreenimine koodiülevaatuseks: Käesolevas töös kasutati mudeleid “nullsõppe” (ingl *zero-shot*) tingimustes. Finantreenimine koodiülevaatuse andmestikul, nagu seda tegid Tufano jt [8], võiks oluliselt parandada tulemusi, eriti Code Llama 70B puhul.

Konteksti laiendamine: Uurida, kuidas mõjutab tulemusi suurema konteksti andmine mudelile — näiteks kogu faili sisu, sõltuvused ja koodiülevaatuse ajalugu.

Multimodaalsed lähenemised: Uurida, kas staatilise analüüsi tööriistade ja LLM-ide kombineerimine (kus staatilise analüüsi leiud esitatakse LLM-ile kontekstina) parandab oluliselt tuvastamistäpsust.

Inimekspertide ja LLM-ide võrdlus: Praegune uuring võrdles ainult LLM-e SonarQube’iga. Huvitav oleks võrrelda LLM-ide tulemusi kogenud inimülevaatajate tulemustega, mis võimaldaks paremini hinnata lõhet automatiseeritud ja manuaalse ülevaatuse vahel.

7. Kokkuvõte

Käesolev bakalaureusetöö uuris suurkeelemudelite sobivust automaatse koodiülevaatuse tööriistadena. Töö eesmärk oli hinnata kolme suurkeelemudelit — GPT-4, Claude 3 Opus ja Code Llama 70B — nende võime poolest tuvastada koodifragmentidest erinevat tüüpi vigu: loogilisi vigu, turvaauke, jõudlusprobleeme ja stiilirikkumisi.

7.1 Uurimisküsimuste kokkuvõte

UK1: Milline on LLM-ide täpsus erinevat tüüpi koodivigade tuvastamisel?

Eksperimendi tulemused näitasid, et LLM-ide täpsus on veatüübist sõltuv. Parim tulemus saavutati stiilirikkumiste tuvastamisel (GPT-4 $F1 = 100\%$) ja kõige keerulisemaks osutus loogiliste vigade tuvastamine (GPT-4 $F1 = 79,3\%$). Turvaaukude tuvastamisel oli GPT-4 eriti tugev ($F1 = 91,7\%$). Üldise keskmise järgi saavutasid GPT-4, Claude 3 Opus ja Code Llama 70B $F1$ -mõõdikud vastavalt 82,5%, 77,9% ja 70,0%.

UK2: Kuidas erineb eri LLM-ide jõudlus koodiülevaatuse ülesannetes?

GPT-4 saavutas parimad tulemused kõigis kategooriates, millele järgnes Claude 3 Opus sarnase jõudlusega. Code Llama 70B jäi mõlemale alla, eriti Java koodi ja turvaaukude tuvastamise osas. Suurim erinevus mudelite vahel ilmnis turvaaukude tuvastamisel (GPT-4 91,7% vs Code Llama 70B 58,3%).

UK3: Milliseid vigade tüüpe suudavad LLM-id kõige paremini ja kõige halvemini tuvastada?

Kõik mudelid said kõige paremini hakkama stiilirikkumiste tuvastamisega, kuna need on hästi defineeritud ja mustrikordsed. Kõige raskemaks osutusid loogilised vead, eriti need, mis avalduvad ainult äärejuhtudel. Turvaaukude tuvastamisel erinesid mudelite tulemused kõige enam, mis viitab sellele, et see on valdkond, kus mudeli treeninguandmestiku sisu mängib olulist rolli.

UK4: Kuidas võrreldakse LLM-ide automaatne koodiülevaatus traditsiooniliste staatilise analüüsi tööriistadega?

Kõik kolm LLM-i ületasid SonarQube'i üldise $F1$ -mõõdiku poolest (SonarQube $F1 = 57,6\%$). LLM-ide peamine eelis on kontekstuaalse mõistmise võime ja looduskeelsete seletuste genereerimine. SonarQube on endiselt konkurentsivõimeline stiilirikkumiste ja teatud turvaaukude tuvastamisel tänu madalamale valepositiivsete määrale ja kiirele töötlusele.

7.2 Peamised järeldused

Käesoleva uurimistöö peamised järeldused on järgmised.

Esiteks, suurkeelemudelid on koodiülevaatuse tööriistadena paljulubavad ja ületavad traditsioonilist staatilise analüüsi loogiliste vigade ja jõudlusprobleemide tuvastamises. Teiseks, kõigi uuritud LLM-ide seas on GPT-4 hetkel parim üldotstarbelises koodiülevaatuses, eriti turvaaukude tuvastamises. Kolmandaks, avatud lähtekoodiga Code Llama 70B pakub madalamaid tulemusi, kuid on väärtuslik privaatsuskaalutlustest tingitud keskkondades. Neljandaks, LLM-id ja staatilise analüüsi tööriistad täiendavad üksteist ning optimaalne lahendus kombineerib mõlemat. Viiendaks, hallutsinatsioon on endiselt oluline piirang, mis eeldab inimese järelevalvet LLM-põhises koodiülevaatuses.

7.3 Töö panus

Käesoleva bakalaureusetöö peamine panus on esimene süstemaatiline võrdlev uuring kolme kaasaegse suurkeelemudeli vahel koodiülevaatuse kontekstis. Töö pakub standardiseeritud hindamise raamistikku, mida saab kasutada tulevikus uute mudelite hindamiseks. Lisaks tuvastab töö selged soovitusel, milliste veatüüpide puhul LLM-id on kõige kasulikumad, mis aitab tarkvaraorganisatsioonidel teha teadlikke otsuseid LLM-ide integreerimisel koodiülevaatuse protsessi.

Uurimistulemused on kooskõlas laiema trendiga tarkvaratehnika valdkonnas, kus LLM-id on hakanud mängima üha olulisemat rolli arendajate abistamisel. Samas rõhutab käesolev töö, et neid tööriistu tuleb kasutada kriitilise meelega ning manuaalse inimülevaatuse täieliku asendamine LLM-idega ei ole praeguses arengujärgus soovitatav.

References

- [1] Bacchelli A. and Bird C. Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*. IEEE Press, 2013, pp. 712–721. DOI: [.](#)
- [2] McIntosh S., Kamei Y., Adams B., and Hassan A. E. An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering* 21.5 (2016), pp. 2146–2189. DOI: [.](#)
- [3] Sadowski C., Söderberg E., Church L., Sipko M., and Bacchelli A. Modern Code Review: A Case Study at Google. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM, 2018, pp. 181–190. DOI: [.](#)
- [4] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). [.](#)
- [5] Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. Tech. rep. Anthropic, 2024. [.](#)
- [6] Rozière B., Gehring J., Gloeckle F., Sootla S., Gat I., Tan X. E., Adi Y., Liu J., Sauvestre R., Remez T., Rapin J., Kozhevnikov A., Evtimov I., Bitton J., Bhatt M., Ferrer C. C., Grattafiori A., Xiong W., Défossez A., Copet J., Azhar F., Touvron H., Martin L., Usunier N., Scialom T., and Synnaeve G. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023). [.](#)
- [7] GitHub. GitHub Copilot: Your AI Pair Programmer. *GitHub Documentation* (2023). [.](#)
- [8] Tufano M., Ni S. Y., Chromik S., Brunner R., Svyatkovskiy A., Clement C. B., and Sundaresan N. Towards Automating Code Review Activities. *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1162–1174. DOI: [.](#)
- [9] Devlin J., Chang M.-W., Lee K., and Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2019). [.](#)
- [10] Guo D., Lu S., Duan N., Wang Y., Zhou M., and Yin J. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. *arXiv preprint arXiv:2203.03850* (2022). [.](#)
- [11] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser Ł., and Polosukhin I. Attention Is All You Need. *Advances in Neural Information Processing Systems* 30 (2017). [.](#)

- [12] Ouyang L., Wu J., Jiang X., Almeida D., Wainwright C. L., Mishkin P., Zhang C., Agarwal S., Slama K., Ray A., Schulman J., Hilton J., Kelton F., Miller L., Simens M., Aspell A., Welinder P., Christiano P. F., Leike J., and Lowe R. Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744. .
- [13] Chen M., Tworek J., Jun H., Yuan Q., Oliveira Pinto H. P. de, Kaplan J., Edwards H., Burda Y., Joseph N., Brockman G., Ray A., Puri R., Krueger G., Petrov M., Khlaaf H., Sastry G., Mishkin P., Chan B., Gray S., Ryder N., Pavlov M., Power A., Kaiser L., Bavarian M., Winter C., Tillet P., Such F. P., Cummings D., Plappert M., Chantzis F., Barnes E., Herbert-Voss A., Guss W. H., Nichol A., Paino A., Tezak N., Tang J., Babuschkin I., Balaji S., Jain S., Saunders W., Hesse C., Carr A. N., Leike J., Achiam J., Misra V., Morikawa E., Radford A., Knight M., Brundage M., Murati M., Mayer K., Welinder P., McGrew B., Amodei D., McCandlish S., Sutskever I., and Zaremba W. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). .
- [14] Liu J., Xia C. S., Wang Y., and Zhang L. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Advances in Neural Information Processing Systems*. Vol. 36. 2023.
- [15] Just R., Jalali D., and Ernst M. D. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2014, pp. 437–440. DOI: .
- [16] Jiang J., Huang L., Liu Z., Lo D., and Xia X. Impact of Code Language Models on Automated Program Repair. *arXiv preprint arXiv:2302.05020* (2023). .
- [17] Wang Y., Le H., Gotmare A. D., Bui N. D., Li J., and Hoi S. C. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv preprint arXiv:2305.07922* (2023). .
- [18] Lu S., Guo D., Ren S., Huang J., Svyatkovskiy A., Blanco A., Clement C. B., Drain D., Jiang D., Tang D., Li G., Zhou L., Shen L., Zhou L., Tufano M., Gong M., Zhou M., Duan N., Sundaresan N., Ni S. Y., Chakraborty S., and Chen W. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. *arXiv preprint arXiv:2102.04664* (2021). .

- [19] Hellendoorn V. J., Sutherland J., Devanbu P., and Bacchelli A. Towards Automating Code Review at Scale. *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2021, pp. 1479–1482. DOI: [.](#)
- [20] Fan A., Gokkaya B., Harman M., Lyubarskiy M., Sengupta S., Yoo S., and Zhang J. M. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint arXiv:2310.03533* (2023). [.](#)
- [21] Zhang T., Bosnic I., Ieu I. R., Aydın D., and Lo D. Using Pre-trained Models to Boost Code Review Automation. *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 2022, pp. 1–12.
- [22] Nijkamp E., Pang B., Hayashi H., Tu L., Wang H., Zhou Y., Savarese S., and Xiong C. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *arXiv preprint arXiv:2203.13474* (2022). [.](#)
- [23] Pornprasit C. and Tantithamthavorn C. JITBot: An Explainable Just-In-Time Defect Prediction Bot. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2022.

Lisad

Lisa A: Näidiskoodifragmendid testkogumist

Käesolev lisa esitab representatiivsed näited testkogumist igast veatüübist.

Lisa A.1: Loogilise vea näide (Python)

Järgmine Pythoni koodifragment sisaldab off-by-one viga kahendotsingus (ingl *binary search*).

Viga asub real 7, kus `right = mid` peaks olema `right = mid - 1`.

```
def binary_search(arr, target):
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid # VIGA: peaks olema right = mid - 1
    return -1
```

GPT-4, Claude 3 Opus ja Code Llama 70B tuvastasid kõik selle vea.

Lisa A.2: Turvaaugu näide (Python)

Järgmine koodifragment sisaldab SQL-i süstimise turvaugu.

```
import sqlite3

def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    # VIGA: kasutajasisend ühendatakse otse SQL-päringuga
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    cursor.execute(query)
    return cursor.fetchone()
```

Korrektne lähenemine oleks kasutada parameetriseeritud päringut:

```
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))
```

GPT-4 ja Claude 3 Opus tuvastasid turvaauku, selgitasid SQL-i süstimise ründeviisi ja pakkusid parameetriseeritud päringu lahenduse. Code Llama 70B tuvastas turvaauku, kuid ei pakkunud üksikasjalikku selgitust.

Lisa A.3: Jõudlusprobleem (Java)

Järgmine Java koodifragment demonstreerib ebatõhusat StringBuilder'i kasutamist tsüklis.

```
public String buildString(List<String> parts) {
    String result = "";
    for (String part : parts) {
        // VIGA: String concatenation tsüklis on  $O(n^2)$ 
        result = result + part + ", ";
    }
    return result.substring(0, result.length() - 2);
}
```

Korrektne lähenemine oleks kasutada StringBuilder'i:

```
StringBuilder sb = new StringBuilder();
for (String part : parts) {
    sb.append(part).append(", ");
}
return sb.toString().substring(0, sb.length() - 2);
```

Lisa B: Eksperimendi andmestik

Käesoleva eksperimendi kõik 120 koodifragmenti koos annotatsioonidega on kättesaadavad veebiaadressil <https://github.com/example/llm-code-review-dataset> (hüpoteetiline, näitlikustamise eesmärgil).

Lisa C: Viipade täistekst

Lisa C.1: Standardne koodiülevaatuse viip

You are an expert code reviewer with deep knowledge of software security, performance optimization, and coding best practices. Analyze the following **{language}** code snippet and identify any issues.

For each issue found, provide:

- **Issue type:** One of [Logic Error / Security Vulnerability / Performance Issue / Style Violation]
- **Location:** Line number(s) where the issue occurs
- **Description:** Clear explanation of why this is an issue
- **Suggested fix:** How to correct the issue

If no issues are found, state "No issues found" and briefly explain why the code is correct.

Code: **{code_snippet}**

Lisa D: SonarQube seadistus

SonarQube analüüs viidi läbi järgmise seadistusega:

- **Versioon:** SonarQube Community Edition 10.3
- **Python reeglite komplekt:** Sonar way (vaikimisi)
- **Java reeglite komplekt:** Sonar way (vaikimisi)
- **Analüüsi tüüp:** Kohaliku faili analüüs (sonar - scanner)
- **Minimaalse raskusastme lävi:** LOW (kõik probleemid)

Litsents

Lihtlitsents lõputöö reprodutseerimiseks ja lõputöö üldsusele kättesaadavaks tegemiseks

Mina, **Uku Renek** (sünnikuupäev: XX.XX.XXXX),

1. annan Tartu Ülikoolile tasuta loa (lihtlitsentsi) enda loodud teose

„Suurkeelemudelite kasutamine automaatses koodiülevaatuses”,

mille juhendaja on prof. Vanem Juhendaja,

(a) reprodutseerimiseks säilitamise ja üldsusele kättesaadavaks tegemise eesmärgil, sealhulgas digitaalarhiivi DSpace-is lisamise eesmärgil kuni autoriõiguse kehtivuse tähtaja lõppemiseni;

(b) üldsusele kättesaadavaks tegemiseks Tartu Ülikooli veebikeskkonna kaudu, sealhulgas digitaalarhiivi DSpace'i kaudu kuni autoriõiguse kehtivuse tähtaja lõppemiseni.

2. olen teadlik, et punktis 1 nimetatud õigused jäävad alles ka autorile.

3. kinnitan, et lihtlitsentsi andmisega ei rikuta teiste isikute intellektuaalomandi ega isikuandmete kaitse seadusest tulenevaid õigusi.

Tartus, XX.XX.2025 (kuupäev)